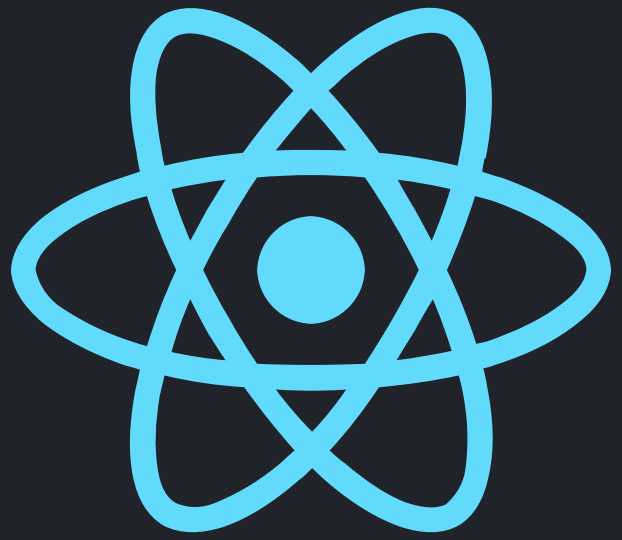

REACT NATIVE CLI

What Is React Native?

- **JavaScript framework** for building native mobile apps using React concepts (components, props, state)
- Compiles to **native Android/iOS binaries**, giving better performance than pure WebView approaches
- In this tutorial, you will set up React Native CLI on Windows, Ubuntu, and macOS, then build and run a simple Android app

React Native CLI vs Expo

Expo (Managed)

- Faster onboarding, over-the-air (OTA) updates out of the box
- Many SDKs and libraries preconfigured and ready to use
- Less native configuration access by default (though improved with Expo Prebuild)

React Native CLI (Bare Metal)

- Closer to native Android/iOS projects (Gradle and Xcode)
- Full control of native modules, build variants (debug/release), and configs
- Ideal for learning internals and integrating with existing native apps

Common Prerequisites (All OS)

- **Node.js** + npm or Yarn
- **JDK 17** (recommended)
- **Android Studio** + Android SDK
- Android **device or emulator** (AVD)
- Android SDK must include: **Platform** (e.g., Android 14 / API 34), **SDK Build-Tools**, **Android Emulator**, and **Platform-Tools**

Windows: Install Node, JDK, Android Studio

- Install **Node.js LTS** from the official website (nodejs.org)
- Verify installation in PowerShell or Command Prompt:

```
$ node -v
$ npm -v
```

- Install **JDK 17** (e.g., Eclipse Temurin or Oracle)
- Install **Android Studio** (Ensure you include Android SDK, Platform-Tools, and Android Emulator during setup)

Windows: Configure Environment Variables

- Set system variables (System Properties → Environment Variables):
- **JAVA_HOME** → JDK install path (e.g., `C:\Program Files\Eclipse Adoptium\jdk-17...`)
- **ANDROID_HOME** → Android SDK path (e.g., `C:\Users\<you>\AppData\Local\Android\Sdk`)
- Add the following to your **Path** variable:

```
%ANDROID_HOME%\platform-tools
%ANDROID_HOME%\emulator
%ANDROID_HOME%\tools
%ANDROID_HOME%\tools\bin
```

Ubuntu: Install Node, JDK, Android SDK

- Update package lists and install required tools via apt:

```
$ sudo apt update
$ sudo apt install -y nodejs npm openjdk-17-jdk adb
```

- Install **Android Studio** (tar.gz package) or command-line SDK tools
- Set your suggested SDK path under: **`~/Android/Sdk`**

Ubuntu: Configure Environment Variables

- Add the following lines to your **~/.bashrc** or **~/.zshrc** file:

```
export JAVA_HOME="$HOME/jdk-17" # or system JDK path

export ANDROID_HOME=$HOME/Android/Sdk

export PATH=$PATH:$ANDROID_HOME/emulator:$ANDROID_HOME/tools:
$ANDROID_HOME/tools/bin:$ANDROID_HOME/platform-tools
```

macOS: Install Tools & Configure Env

- Install **Homebrew** (if needed), then install Node, Watchman, JDK 17, and Android Studio + SDK.
- Configure your environment variables in **~/.zshrc** or **~/.bashrc**.

```
# Install Homebrew (if not already installed)
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install required tools and JDK
$ brew install node watchman
$ brew install openjdk@17

# Add the following to your ~/.zshrc or ~/.bashrc
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export ANDROID_HOME=$HOME/Library/Android/sdk
export
PATH=$PATH:$ANDROID_HOME/emulator:$ANDROID_HOME/tools:$ANDROID_HOME/tools/bin:$ANDROID_HOME/platform-tools
```

Install React Native CLI and Create Project

- **Preferred:** Use the community CLI via npx (creates project instantly)

```
$ npx @react-native-community/cli init MyFirstApp
```

- **Optional:** Install CLI globally (older style)

```
$ npm install -g react-native-cli
$ react-native init MyFirstApp
```

Project Structure Overview

- **`android/`** (Gradle-based Android project)
- **`ios/`** (Xcode project)
- **`app.json`**, **`package.json`**, **`index.js`**, **`babel.config.js`**
- Main JS code lives in **`App.js`** or **`index.js`**
- Android build config lives under **`android/`** (Gradle)

Android Emulator/Device Setup + Run

- In Android Studio, create and start an **AVD** (recent API level)
- For physical device: enable Developer Options + USB debugging

```
$ adb devices
```

- From the project root, start Metro and run the app:

```
$ npx react-native start # start Metro
$ npx react-native run-android # build, install, launch
```

Key React Native CLI Commands

- **Project lifecycle:** Start bundler, build, and deploy

```
$ npx react-native start # start Metro bundler
$ npx react-native run-android # build & deploy debug
$ npx react-native run-android --variant=release # local release build
```

- **Utilities:** View and manage logs

```
$ npx react-native log-android # view RN Android logs
```

Gradle Tasks: Build, Install, Clean, Test

- From the `android/` directory, use the Gradle wrapper:

```
$ cd android
$ ./gradlew tasks # (Windows: gradlew.bat tasks)
$ ./gradlew assembleDebug # build debug APK
$ ./gradlew assembleRelease # build release APK
$ ./gradlew installDebug # install debug on device/emulator
$ ./gradlew clean # clean build outputs
$ ./gradlew test # run unit tests (if configured)
```

Build a Simple Counter App (Concept)

- In **`App.js`**, create a functional component using **`useState`** for count
- **Layout:** `SafeAreaView` → `View` → `Text` (title) + `Text` (count) + `Button/Pressable` (increment)
- **Focus:** `import React/useState`, define `function App()`, `export default App`
- Keep it **simple** so learners can type it themselves while teaching.

Live Reloading and Debugging

- Enable **Fast Refresh** via the in-app developer menu (shake device, or Ctrl+M on Windows/Linux, Cmd+M on macOS emulator)
- UI updates instantly after saving code
- Use `console.log` for JS debugging

```
$ npx react-native log-android   # view Android logs
```

Note: Or use Android Studio Logcat for advanced native and JS logs.

Common Errors and Fixes

SDK location not found / No build tools

Android Studio cannot find the required SDK or build components to compile the app.

Set **`ANDROID_HOME`** correctly and install **SDK Platform + Build-Tools** in Android Studio.

Unsupported Java version

"Java 1.8 not supported" or similar errors when Gradle attempts to build the Android project.

Ensure **JDK 17** is installed and your **`JAVA_HOME`** environment variable points to it.

Metro not running / red screen

The app installs but shows a red error screen stating it cannot connect to the development server.

Run **`npx react-native start`** in a separate terminal and ensure your device is on the same network.

Next Steps for Beginners

- **Add more UI:** inputs, lists, navigation (e.g., React Navigation via `npm install @react-navigation/native`)
- **Explore platform features:** permissions, camera, storage via native modules
- **Learn Gradle config:** `minSdkVersion`, `targetSdkVersion`, and signing configs in `android/app/build.gradle`
- **Practice the workflow:** edit App.js → save → Fast Refresh → build release with `./gradlew assembleRelease`

App Entry: index.js

```
// index.js
import { AppRegistry } from 'react-native';
import App from './App';
import { name as appName } from './app.json';

AppRegistry.registerComponent(appName, () => App);
```

- **AppRegistry.registerComponent** tells React Native which root component to load for the native app entry point
- **App** is your main React component, defined in App.js in the same folder

Basic App Component + Styles (Counter UI)

- **`useState(0)`** creates state; pressing Button updates count and triggers re-render
- **`StyleSheet`** uses Flexbox to center content vertically and horizontally

```
// App.js
import React, { useState } from 'react';
import { SafeAreaView, View, Text, Button, StyleSheet } from 'react-native';

function App() {
  const [count, setCount] = useState(0);
  return (
    <SafeAreaView style={styles.container}>
<View>
<Text style={styles.title}>My First React Native App</Text>
<Text style={styles.counterLabel}>You pressed the button:</Text>
<Text style={styles.counterValue}>{count} times</Text>
<Button title="Press me" onPress={() => setCount(count + 1)} />
</View>
</SafeAreaView>
  );
}
export default App;

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', backgroundColor: '#F2F2F2' },
  title: { fontSize: 24, fontWeight: '600', marginBottom: 16, textAlign: 'center' },
  counterLabel: { fontSize: 18, marginBottom: 8, textAlign: 'center' },
  counterValue: { fontSize: 32, fontWeight: 'bold', marginBottom: 16, textAlign: 'center', color: '#007AFF' },
});
```

Running and Testing Your First App

- **Step 1:** Start Metro in project root:

```
$ npx react-native start
```

- **Step 2:** In another terminal, build and run on Android:

```
$ npx react-native run-android
```

🚀 **After install you should see the title and button — tap "Press me" and watch the counter increase in real time.**